

Smart Stock Analyzer: A DBMS-Driven Portfolio Management and Risk-Aware Recommendation System with Operating-System-Level Concurrency Control

Hariom Raturi (24021325), **Paarth Joshi** (240212029), **Suryadev Singh** (240212376)

Team Code Geeks, Project OSDBMS-V-2026-T139

hariomraturi093@gmail.com | paarthjoshi3120@gmail.com | snnsurya31@gmail.com

Abstract

Recent research on stock markets is dominated by deep-learning models that forecast prices, yet the data-management layer that records trades, protects portfolio state and serves results to many simultaneous users receives almost no attention in the surveyed literature. This paper presents the Smart Stock Analyzer, a student-oriented platform whose contribution lies in that layer. The system couples a normalized relational schema (Users, Stocks, Portfolio, Transactions, Logs) with an analytics module that computes holdings value, profit/loss and break-even price, and a transparent risk-score decision engine that maps a score in $[0,1]$ to four graded responses. Operating-system mechanisms (worker threads, mutual exclusion, bounded resource pools) keep concurrent trading correct. We implemented a working prototype on SQLite and Python and ran three experiments. (i) With 16 concurrent client threads, an unsynchronised check-then-update trade path produced 6,580 inconsistent shares on average, whereas the atomic, lock-protected path produced none in all 25 runs. (ii) A single index on the Transactions table reduced portfolio-query latency by 18–28 times for tables of 10,000 to 500,000 rows. (iii) A synthetic 200-stock study showed that the decision engine places stocks in all four response bands. We discuss limitations, notably the use of synthetic market data, and outline future work. The system is an educational demonstrator and does not provide financial advice.

Keywords: stock analysis; portfolio management; relational database; ACID transactions; concurrency control; mutual exclusion; risk scoring; recommendation system; explainability.

1. Introduction

Investment decisions depend on current and historical prices, the risk profile of each security, the composition of the investor's portfolio and the return on past trades. For beginners, gathering this information from several sources is slow, and no single place shows what was bought, what it is worth now and how each trade changed the overall position. Professional platforms solve part of this problem but are large, complex and frequently dependent on external services, which makes them poorly suited for students who want to learn how such systems actually work.

The scientific literature on markets has moved quickly in the last three years. Systematic reviews now cover deep-learning forecasting [1], [6], algorithmic trading [2], asset management [3], portfolio management [9], [10] and feature selection for neural forecasters [4]. A common thread is the pursuit of better predictive accuracy. Far less is said about the engineering substrate on which any forecaster must run: a transactional store that never loses or double-counts a trade, a portfolio view that stays consistent when many users act at once, and decision logic that can be explained to the person using it. These are classical database and operating-system questions [11], [12], and they are the focus of this work.

Research questions. We ask three questions. **RQ1:** How much data inconsistency appears when concurrent buy/sell requests are processed without synchronisation, and does an atomic, lock-protected transaction eliminate it? **RQ2:** How does a conventional index affect the latency of the portfolio queries that dominate the workload as the transaction history grows? **RQ3:** Can a transparent, threshold-based decision engine over a normalised risk score produce graded and auditable responses?

Contributions. (1) A modular architecture and relational schema for a small but complete stock-analysis platform. (2) A closed-form analytics layer for holdings value, average cost, profit/loss and break-even price. (3) A four-band risk decision engine whose thresholds are explicit constants. (4) A mapping of core OS concepts (threads, mutual exclusion, deadlock avoidance, resource pools) onto trade processing. (5) An empirical evaluation of a working prototype that answers RQ1-RQ3 with measured data.

2. Literature Review

2.1 Deep-learning forecasting. Akşehir and Kılıç reviewed deep-learning stock-forecasting studies from 2020 to 2024 and organised them around seven critical steps of a prediction pipeline, among them feature selection, feature extraction, denoising, sliding-window construction and trading simulation, noting that each step measurably affects model performance [1]. Bhuiyan et al. surveyed deep learning for algorithmic trading, covering recurrent, LSTM, convolutional and hybrid architectures, and identified data noise, overfitting and interpretability as persistent obstacles [2]. A 2026 review traces the progression from LSTM to Transformer models and adds computational efficiency to the list of challenges, pointing toward multimodal fusion, explainable AI and real-time adaptive learning [6]. Sarıkoç examined whether feature selection actually helps deep learners, analysing work from 2010 to 2025 and classifying methods as filter, wrapper, embedded and hybrid [4].

2.2 Landscape of techniques and data. Chaudhari and Purswani followed a PRISMA-based procedure and reported that the reviewed literature was 7% statistical, 9% machine learning, 23% deep learning, 20% hybrid, 25% a combination of machine learning and deep learning, and 14% spanning several categories; they also found that Yahoo! was the most common source of trading data and that sentiment data, mostly tweets, improved prediction [5]. We reproduce this distribution in Fig. 2(a) because it shows how heavily the field has shifted to learning-based models, and how common free public price feeds are. Both facts shape our design: the prototype reads prices from a replaceable feed rather than assuming a particular provider.

2.3 Portfolio management and explainability. Reis, Serra and Gama examined deep learning in financial asset management between 2018 and 2023 and highlighted explainable AI and deep reinforcement learning (DRL) as emerging directions [3]. A review of deep learning in investment portfolio management found a strong dominance of reinforcement-learning algorithms because of their ability to act through time [10], and a literature review of AI in portfolio management argued that explainability could increase transparency toward stakeholders [9]. Kumar et al. discuss DRL for dynamic portfolio optimisation, proposing transfer learning, explainable reinforcement learning, hybrids with traditional methods and robust risk management as future directions [7]. Guan and Liu proposed an empirical approach that uses integrated gradients to explain what DRL portfolio agents have learned, motivated by the black-box nature of deep networks [8].

2.4 Research gap. Taken together, these works concentrate on the model: what to predict, with which features and with what explanation. None of the surveyed studies is primarily about the transactional layer that must record trades correctly under concurrency, nor about the indexing and schema choices that keep portfolio queries fast. The reviews' repeated call for interpretability [2], [3], [6], [9] is, however, directly relevant to our decision engine, which we deliberately keep rule-based so that every response can be traced to a score and a threshold.

Work	Focus	Main finding used here	Data-management layer?
[1] Akşehir & Kılıç, 2024	DL forecasting pipeline	Seven pipeline steps affect accuracy	Not addressed
[2] Bhuiyan et al., 2025	DL for algorithmic trading	Noise, overfitting, interpretability	Not addressed
[3] Reis et al., 2025	DL in asset management	XAI and DRL emerging	Not addressed

Work	Focus	Main finding used here	Data-management layer?
[5] Chaudhari & Purswani	AI prediction review	Technique mix; public data dominant	Not addressed
[7] Kumar et al., 2025	DRL portfolio optimisation	Needs risk management and XAI	Not addressed
[8] Guan & Liu, 2021	Explainable DRL	Integrated-gradient explanations	Not addressed
This work	Transactional DB + OS layer	Measured consistency, latency, decisions	Yes

Table 1. Positioning of this work against recent literature.

3. Problem Formulation

Let U be the set of users, K the set of stocks, and $x_{u,k}$ the number of shares of stock k held by user u . Each stock has a price $p_k(t)$ and a finite available quantity a_k . A buy order (u,k,q) is valid if $q > 0$ and $q \leq a_k$; a sell order is valid if $q > 0$ and $q \leq x_{u,k}$. The system must satisfy two invariants at all times and for any interleaving of concurrent orders:

- **Conservation:** for every stock, initial quantity = available quantity + shares held by users, and the sum of quantities recorded in the Transactions ledger equals the net shares moved.
- **Non-negativity:** $a_k \geq 0$ and $x_{u,k} \geq 0$, i.e., no overselling and no short positions.

The problem is to design a platform that (a) keeps both invariants under concurrent load, (b) answers portfolio queries quickly as history grows, and (c) turns a computed risk estimate into a clear, auditable response for the user.

4. System Architecture

The system is organised in layers (Fig. 1). The investor interacts through registration and login, stock and market-data views, buy/sell actions and portfolio/history access. Requests travel over HTTP/HTTPS to a web or desktop frontend that performs input validation and displays results; the frontend calls the application server through API calls. The application server contains the business logic: authentication and authorisation, stock and portfolio services, transaction handling and the recommendation engine. It reads and writes the relational database, whose principal tables are Users, Stocks, Portfolio, Transactions and Logs. A data-processing and analysis module (stock analysis, risk assessment, recommendation generation) feeds a decision engine implemented as if/else logic, whose four possible outputs converge on a dashboard showing the portfolio summary, profit/loss analysis, recommendations and transaction history. Every outcome is written to the Logs table.

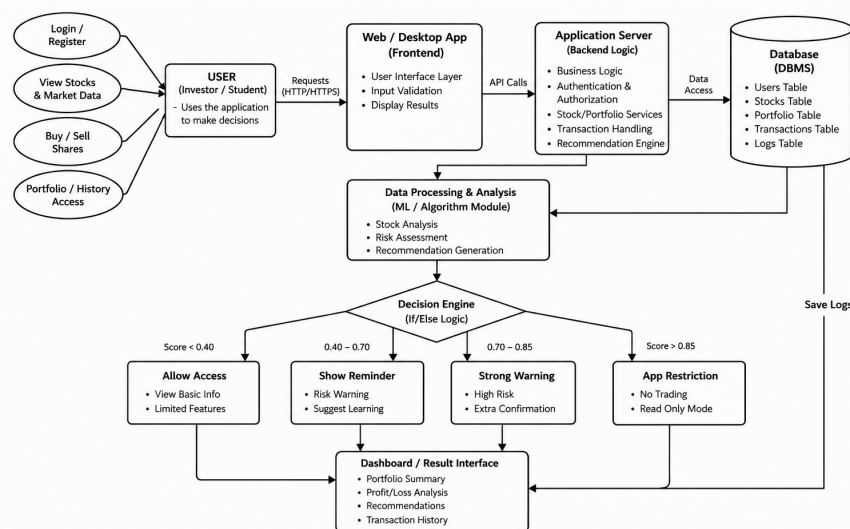


Fig. 1. High-level architecture of the Smart Stock Analyzer.

5. Database Design

5.1 Schema. The schema is in third normal form: each fact is stored once and tables are linked by keys. The Portfolio table resolves the many-to-many relationship between users and stocks and carries the quantity and average purchase price of each holding. Transactions is an append-only ledger. Integrity is enforced in the database through primary keys, a unique constraint on e-mail and symbol, foreign keys, and CHECK constraints ($\text{price} > 0$, $\text{quantity} > 0$, $\text{available quantity} \geq 0$), so invalid states are rejected even if application code is faulty.

Table	Key attributes	Role
Users	user_id (PK), name, email (UNIQUE), password_hash	Accounts and authentication
Stocks	stock_id (PK), symbol (UNIQUE), price (>0), available_qty (≥ 0)	Master data and stock availability
Portfolio	(user_id, stock_id) composite PK, quantity (≥ 0), avg_price	Current holdings
Transactions	txn_id (PK), user_id, stock_id, type (BUY/SELL), quantity (>0), price, ts	Append-only trade ledger
Logs	log_id (PK), user_id, event, risk_score, created_at	Audit trail of decisions

Table 2. Relational schema of the prototype.

5.2 Atomic trade transaction. A buy is one ACID transaction [12] with four steps: validate the order; decrease *available_qty*; insert the ledger row; upsert the portfolio holding with a recomputed average cost. The transaction either commits as a whole or rolls back, so a failure between steps cannot leave the ledger and the stock quantity disagreeing.

5.3 Indexing. The dominant read is “show this user's holdings and trade summary”, which filters the Transactions table by user. Without an index this is a full table scan whose cost grows linearly with the history. We therefore create a B-tree index on Transactions(user_id). Section 8 quantifies the effect.

6. Analytics and Risk Decision Engine

6.1 Portfolio mathematics. For a holding of q shares with average cost c and current price p , the system computes:

$$\text{Value } V = q \cdot p \quad \text{Profit/Loss } PL = (p - c) \cdot q \quad \text{Return } R = (p - c)/c \times 100\% \quad (1)$$

After a further purchase of q_n shares at price p_n , the average cost is updated as the weighted mean:

$$c' = (q \cdot c + q_n \cdot p_n) / (q + q_n) \quad (2)$$

The break-even price including total charges f is $BEP = c + f/q$ (3). Fig. 3(b) shows these quantities for a simulated holding with three purchases: the average-cost line steps with each buy, and the shaded area between price and cost is profit or loss. In that run the final price was 116.06 against an average cost of 112.03, giving a profit of 141.06 on 35 shares, which matches Eq. (1).

6.2 Risk score. The prototype defines a normalised risk score in $[0,1]$ from the annualised volatility σ of daily log-returns and the maximum drawdown D over the trailing year:

$$s = 0.5 \cdot \min(\sigma / 0.8, 1) + 0.5 \cdot \min(D / 0.6, 1) \quad (4)$$

The weights and caps are design constants chosen for transparency and can be tuned without touching other code. The score is deliberately simple; it is a stand-in that the learning-based risk models discussed in Section 2 could later replace behind the same interface.

6.3 Decision engine. The score is mapped to a response by threshold rules (Table 3). Because the rules are explicit, each decision recorded in Logs can be reproduced from the stored score, which addresses the interpretability concern raised repeatedly in the literature [2], [3], [6], [9].

Score s	Response	Behaviour
< 0.40	Allow access	Basic information; limited features

Score s	Response	Behaviour
0.40 - 0.70	Show reminder	Risk warning; suggest learning material
0.70 - 0.85	Strong warning	High-risk alert; extra confirmation required
> 0.85	App restriction	Trading disabled; read-only mode

Table 3. Decision-engine thresholds.

7. Operating-System Design

A trading application is inherently concurrent: many users submit orders while a background task refreshes prices. Four OS mechanisms [11] are applied.

Threads and processes. Each client request is handled by its own worker thread so that a slow request does not block others. A background thread refreshes prices and recalculates portfolio values, sleeping between cycles to avoid wasting processor time.

Mutual exclusion. The available quantity of a stock and a user's holdings are shared mutable data. A naive implementation reads the quantity, checks it, and writes a new value in separate steps; two threads can both read the same value and both succeed, a classic lost-update race. The critical section is therefore protected by a write lock acquired at the start of the transaction (in SQLite, BEGIN IMMEDIATE; in a server DBMS, row-level locking such as SELECT ... FOR UPDATE) and the update is expressed relative to the stored value ($available_qty = available_qty - q$) rather than as an absolute value computed outside the lock.

Deadlock avoidance. When an operation needs several locks (for example a stock row and a portfolio row) they are always acquired in a fixed global order, which breaks the circular-wait condition.

Resource management. A bounded pool of workers and database connections limits memory and processor use. Requests arriving when all workers are busy wait in a queue instead of exhausting the machine, which exercises waiting, scheduling and resource sharing in practice.

7A. Implementation Overview

The prototype follows a modular plan in which each module is developed and tested independently before integration. Table 5 summarises the modules, their responsibilities and the verification applied to each. Input validation is performed twice: in the frontend for fast feedback and again in the application server, because client-side checks cannot be trusted. Database constraints provide a third, final barrier.

Module	Responsibility	Verification
Account	Registration, login, password hashing, sessions	Duplicate e-mail and bad-credential tests
Stock view	List stocks, current price, availability	Query correctness; empty-result handling
Transaction	Validated atomic buy/sell, ledger insert	Rollback and concurrency tests (Section 8)
Portfolio	Holdings, average cost, P/L, break-even	Hand-computed examples against Eqs. (1)–(3)
Recommendation	Risk score and decision engine	Threshold-boundary tests (0.40, 0.70, 0.85)
Logging	Audit of trades and decisions	Every decision reproducible from stored score

Table 5. Module responsibilities and verification.

Error handling. Invalid input (non-positive quantity, unknown symbol, selling more than owned) is rejected with an explicit message and leaves the database unchanged. Failures inside a trade trigger a rollback, so the user never observes a half-completed order. Because the ledger is append-only, corrections are made by adding compensating rows rather than editing history, which preserves auditability.

8. Experimental Evaluation

8.1 Setup. We implemented the schema of Table 2 in SQLite and the trade path in Python with one connection per thread. Experiments ran on a single development machine; absolute numbers depend on hardware and storage, so we emphasise relative comparisons. Market prices are **synthetic**, generated by geometric Brownian motion; no real market data or real money is involved.

Experiment 1 (RQ1, concurrency). A single stock starts with 3,000 shares. Each of T threads ($T = 1, 2, 4, 8, 16$) submits 150 buy orders of 1–5 shares. In the *unsynchronised* variant, each thread reads the quantity, yields the processor and then writes back $quantity - q$ with autocommit, and appends a ledger row. In the *atomic* variant, the whole trade runs in one BEGIN IMMEDIATE transaction with a relative update. After each run we compute the inconsistency as $|(initial - final\ available) - \text{sum of ledger quantities}|$, i.e., shares sold on paper but never deducted. Each configuration was repeated five times.

Experiment 2 (RQ2, indexing). The Transactions table was filled with 10,000, 100,000 and 500,000 random rows over 1,000 users. We timed 200 portfolio-summary queries (grouped by stock and type for one user) before and after creating an index on `user_id`.

Experiment 3 (RQ3, decision engine). We generated 200 synthetic one-year price paths with annual volatility drawn uniformly from 10% to 95%, computed Eq. (4) for each and counted the response band. The wide volatility range is chosen to exercise all bands and is not meant to represent a real market.

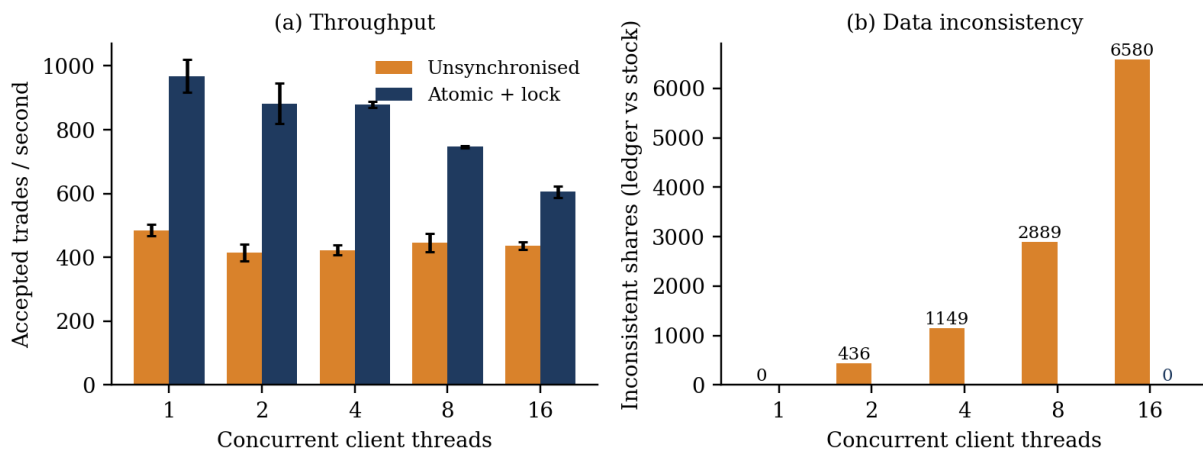


Fig. 2. Concurrency experiment (mean of 5 runs, error bars = s.d.). (a) Accepted trades per second. (b) Shares recorded in the ledger but not deducted from stock.

9. Results and Discussion

9.1 Concurrency (RQ1). With one thread both variants were consistent, as expected. From two threads onwards the unsynchronised variant corrupted state, and the damage grew with contention: on average 436, 1,149, 2,889 and 6,580 shares were inconsistent at 2, 4, 8 and 16 threads (Fig. 2b). The atomic variant produced zero inconsistency in all 25 runs. This answers RQ1: without synchronisation, lost updates appear as soon as two threads overlap and worsen with load, whereas an atomic transaction with a write lock preserves the conservation invariant.

Throughput (Fig. 2a) deserves careful reading. The atomic variant was faster at every thread count (969 versus 485 trades/s at one thread; 605 versus 436 at 16 threads) because it commits one transaction per trade, while the unsynchronised variant, using autocommit, pays a separate durable commit for each statement. The atomic path does slow as threads increase, falling by about 38% from 1 to 16 threads, since write transactions are serialised; this is the price of correctness. We stress that the unsynchronised variant is not a fair speed baseline, but a demonstration of what goes wrong without protection.

Threads	Unsync. trades/s	Atomic trades/s	Unsync. inconsistent shares	Atomic inconsistent shares
1	485	969	0	0
2	415	882	436	0
4	423	879	1,149	0
8	447	746	2,889	0
16	436	605	6,580	0

Table 4. Concurrency results (means of 5 runs).

9.2 Indexing (RQ2). The index cut mean query time from 0.48 ms to 0.026 ms at 10,000 rows (18 $\times$), from 5.71 ms to 0.21 ms at 100,000 rows (28 $\times$) and from 28.0 ms to 1.33 ms at 500,000 rows (21 $\times$) (Fig. 3a). Without the index, latency grew roughly linearly with table size, as expected of a full scan; with it, latency stayed small. Because every dashboard refresh issues such a query, the index is a necessary part of the design rather than an optimisation to add later.

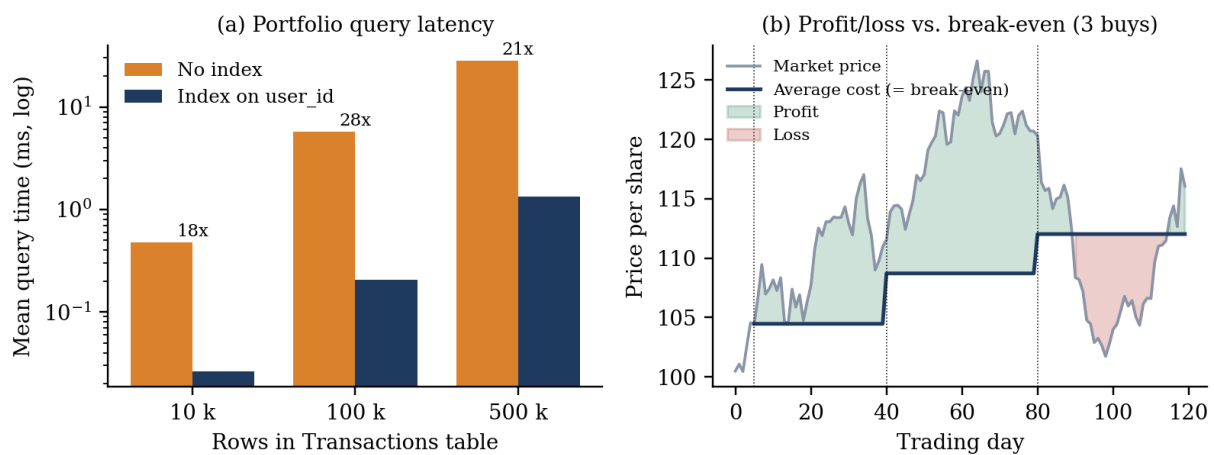
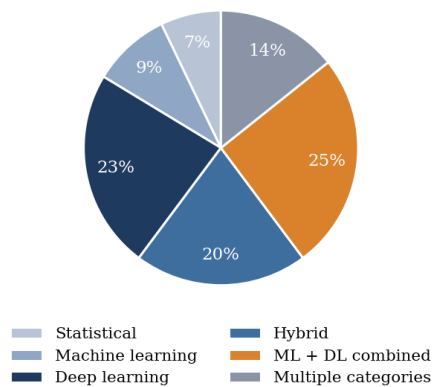


Fig. 3. (a) Portfolio query latency with and without an index (log scale; labels give the speed-up). (b) Simulated profit/loss against average cost (break-even) for three purchases; dotted lines mark the buys.

9.3 Decision engine (RQ3). Of 200 synthetic stocks, 52 (26%) fell in the allow band, 48 (24%) in the reminder band, 32 (16%) in the strong-warning band and 68 (34%) in the restriction band (Fig. 4b); the mean score was 0.65. All four responses are reachable and each decision is reproducible from the stored score. The skew toward the restricted band reflects our deliberately high-volatility synthetic population, not a property of real markets, so these proportions should not be read as market statistics. Fig. 4(a), taken from the literature [5], gives context for why learned risk models may later be worth integrating.

(a) Stock-prediction literature by technique



(b) Decision-engine outcomes (200 synthetic stocks)

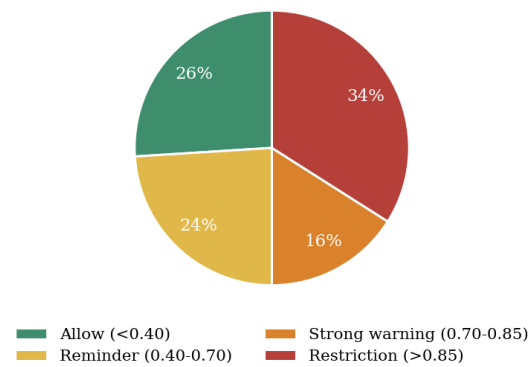


Fig. 4. (a) Distribution of stock-prediction literature by technique, as reported in [5] (percentages as published; they sum to 98% owing to rounding). (b) Decision-engine outcomes for 200 synthetic stocks.

9.4 Discussion and Design Implications

The three experiments, though small, point to a consistent engineering message: correctness and performance in a trading store are decided largely by two inexpensive choices, namely how a trade is wrapped in a transaction and which columns are indexed. Neither requires sophisticated machine learning, and both are invisible to the end user until they fail. This is relevant to the literature in Section 2: highly accurate forecasters [1], [2], [6] still depend on a ledger that is correct, so improvements in prediction cannot compensate for lost updates in the data layer.

Several synchronisation strategies could protect the critical section. Table 6 compares them qualitatively; only the first and last rows were measured in this work, and the others are listed as design alternatives for future benchmarking.

Strategy	Idea	Strength	Weakness
No protection	Read, check, then write separately	Simple	Lost updates; invariants broken (measured)
Application mutex	Process-wide lock around the trade	Easy to reason about	Does not protect multiple server processes
Optimistic version	Version column; retry on conflict	High throughput when conflicts are rare	Retries under contention; more code
DB write lock / row lock	Lock at transaction start; relative update	Correct across processes (measured)	Serialises writers; can cause waiting

Table 6. Synchronisation strategies for the trade critical section (design comparison; only the first and last rows were measured).

Scalability. Because the Portfolio table is keyed by (user, stock), orders from different users on different stocks touch disjoint rows and can proceed in parallel on a DBMS with row-level locking; contention arises only on popular stocks. A production deployment could shard by stock symbol, keep prices in a cache refreshed by the background thread, and move the decision engine into a separate service. These steps follow directly from the OS principles of Section 7 and are natural extensions of the present prototype.

Reproducibility. All experiments use fixed random seeds and an in-process SQLite database file recreated before every run, so the same code can be re-run on any machine with Python. Each concurrency configuration was repeated five times and both the mean and standard deviation are reported; the inconsistency metric is computed directly from the database after the run, not from application counters, so it cannot be affected by the same race it measures. We recommend that future work also report the 95% confidence interval of each mean when more repetitions are available.

Relation to prior work. The reviews surveyed in Section 2 evaluate models on prediction error or trading return. Our metrics are different in kind: ledger consistency, query latency and decision reproducibility. They are complementary rather than competing, and a complete platform would report both, for instance by running a learned forecaster inside the transactional shell described here and measuring end-to-end correctness as well as forecast accuracy.

Educational value. A further outcome is pedagogical: because the unsynchronised and atomic variants differ by a few lines of code yet differ by thousands of corrupted shares, the experiment makes an abstract lesson about race conditions concrete and repeatable for other students.

10. Limitations and Threats to Validity

Synthetic data. Prices and risk populations are simulated, so the decision-band proportions and the profit/loss illustration say nothing about real returns. **Single-node prototype.** SQLite serialises writers, so the throughput curve of the atomic path reflects that engine; a server DBMS with row-level locking would scale differently, and we did not measure distributed deployment. **Hardware dependence.** Absolute throughput and latency depend on the test machine; the relative conclusions (inconsistency without locks, benefit of an index) are the robust findings. **Simple risk model.** Eq. (4) uses only volatility and drawdown and has not been validated against realised outcomes. **Scope.** The platform handles no real money, brokers or regulation and must not be used for actual investment decisions.

Ethical and regulatory note. Financial software can influence real decisions. The platform therefore labels every recommendation as educational, shows the score behind each warning, and restricts trading in the highest-risk band. We make no claim of predictive power and advise against use with real capital.

11. Conclusion and Future Work

Recent reviews show a research community focused on increasingly sophisticated predictive models and increasingly concerned with their interpretability [1]–[10]. This paper complements that direction by examining the data-management and operating-system layer beneath such models. The Smart Stock Analyzer combines a normalised relational schema, atomic trade transactions, a transparent risk-score decision engine and OS-level concurrency control. Measurements on a working prototype showed that unsynchronised concurrent trading corrupts state in proportion to contention while the atomic design does not, that a single index yields an 18–28× reduction in portfolio-query latency, and that a threshold-based engine yields graded, reproducible responses.

Future work includes (i) replacing the synthetic feed with real historical data and evaluating the risk score against realised drawdowns; (ii) substituting the rule-based score with a learned model behind the same interface and explaining it with attribution methods in the spirit of [8]; (iii) benchmarking on a multi-writer server DBMS and a distributed deployment; and (iv) adding sentiment inputs, which the literature reports as beneficial [5].

Acknowledgement

The authors thank their mentor, Shailendra Sir, for his guidance and support throughout this project.

References

- [1] A. D. Akşehir and E. Kılıç, “Analyzing the critical steps in deep learning-based stock forecasting: a literature review,” *PeerJ Computer Science*, vol. 10, 2024, doi: 10.7717/peerj-cs.2312.
- [2] M. S. M. Bhuiyan, M. A. Rafi, G. N. Rodrigues, M. N. H. Mir, A. Ishraq, M. F. Mridha, and J. Shin, “Deep learning for algorithmic trading: A systematic review of predictive models and optimization strategies,” *Array*, vol. 26, art. 100390, 2025, doi: 10.1016/j.array.2025.100390.
- [3] P. Reis, A. P. Serra, and J. Gama, “The role of deep learning in financial asset management: A systematic review,” *arXiv:2503.01591*, Mar. 2025.
- [4] F. Sarıkoç, “Feature selection methods on deep learning algorithms for stock price forecasting: A systematic literature review,” *Eng. Appl. Artif. Intell.*, vol. 181, no. 3, pp. 1–30, 2026.

- [5] C. Chaudhari and G. Purswani, "Stock market prediction techniques using artificial intelligence: A systematic review," CHRIST (Deemed to be University) Institutional Repository.
- [6] "Research progress and trends of deep learning in stock price prediction: A systematic review from LSTM to Transformer," *ITM Web Conf.*, vol. 84, art. 02004, 2026 (ATCIDS 2026).
- [7] N. Kumar, P. Tripathi, K. K. Paroha, P. Agarwal, and D. Bhowmik, "Deep reinforcement learning for dynamic portfolio optimization in financial markets," in *New Paradigms of Business Management in the Era of Analytics, Sustainability and Innovation*, Springer, 2025, pp. 249–268.
- [8] M. Guan and X.-Y. Liu, "Explainable deep reinforcement learning for portfolio management: An empirical approach," arXiv:2111.03995, 2021.
- [9] "Enhancing portfolio management using artificial intelligence: literature review," *Front. Artif. Intell.*, 2024, doi: 10.3389/frai.2024.1371502.
- [10] "Deep learning applications in investment portfolio management: a systematic literature review," Bond University Research Portal, published 18 Mar. 2025.
- [11] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.
- [12] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th ed. New York, NY, USA: McGraw-Hill, 2019.